



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251214
Nama Lengkap	Raffiel Gideon Nyolo Nyolo
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

A. Pengenalan dan Mendefinisikan Set

“Set” atau “himpunan” adalah salah satu tipe data Python yang dapat digunakan untuk menyimpan sekumpulan data yang semuanya berbeda. Salah satu sifat set Python adalah:

- Isi dari Set disebut sebagai anggota (member).
- Anggota dari Set harus bersifat immutable. Beberapa tipe data immutable pada Python: integer, float, string, tuple dan lain-lain. Dengan demikian list dan dictionary (mutable) tidak dapat dimasukkan ke dalam Set.
- Set sendiri bersifat mutable, artinya anda dapat menambah atau mengurangi isi dari sebuah Set. Karena itu Set tidak dapat dimasukkan ke dalam Set.

Untuk mendefinisikan Set, ada beberapa cara yang dapat dilakukan yaitu dengan menggunakan notasi {} dan fungsi set() dan untuk mendefinisikan set kosong hanya perlu menggunakan fungsi set(), seperti contoh berikut ini:

```
# dengan menggunakan {}
bilangan_genap = {2, 4, 6, 8, 10, 12}
bilangan_ganjil = {1, 3, 5, 7, 9, 11}
# dengan menggunakan fungsi set()
pernah_ke_bulan = set('Neil Armstrong', 'Buzz Aldrin')
data = {} #ini akan menghasilkan dictionary kosong
```

B. Pengaksesan Set

Set tidak memiliki indeks, karena itu anda tidak dapat mengakses anggota-anggota dari sebuah Set secara langsung menggunakan indeks. Perhatikan contoh program berikut ini:

```
nim = {'71251214', '71251234', '71251225'}
jumlah_nim = len(nim)
print(jumlah_nim)
#tampilkan isi set satu-persatu
for n in nim:
    print(n)
```

Output:

```
3
71251214
71251234
71251225
```

Karena Set tidak memiliki indeks, sehingga tidak ada urutan posisi anggota, output yang dihasilkan urutannya berbeda dengan deklarasi Set sebelumnya. Ini karena posisi anggota tidak penting untuk tipe data Set. Set adalah jenis data yang dapat diubah, yang berarti isinya dapat bertambah atau berkurang. Cara menggunakan fungsi `add()` untuk menambah anggota ke set ditunjukkan dalam program berikut:

```
plat_nomor = set() # definisikan sebuah set kosong
plat_nomor.add('AB 1890 XA') # tambahkan plat nomor 'AB 1890 XA'
plat_nomor.add('AD 6810 MT') # tambahkan plat nomor 'AD 6810 MT'
print(len(plat_nomor)) # jumlah anggota di dalam Set
plat_nomor.add('AB 1890 XA') # tambahkan plat yang sama sekali lagi
for plat in plat_nomor: # tampilkan semua plat nomor
    print(plat)
```

Output:

```
2
AD 6810 MT
AB 1890 XA
```

Set secara otomatis memeriksa duplikasi saat fungsi `add()` digunakan. Jika nilai belum ada, anggota akan ditambahkan; jika sudah ada, anggota tidak akan ditambahkan lagi sehingga tidak perlu melakukan pengecekan manual.

Untuk menghapus anggota set, dapat digunakan fungsi `discard()`, `remove()`, `pop()`, dan `clear()`, yang masing-masing memiliki cara kerja berbeda. Contoh program penghapusan anggota:

```
bilangan_prima = {13, 23, 7, 29, 11, 5} # hapus 5 dari set tersebut
bilangan_prima.remove(5)
print(bilangan_prima) # hapus 97 (tidak ada)
bilangan_prima.discard(97)
print(bilangan_prima) # ambil dan hapus salah satu
bilangan = bilangan_prima.pop()
print(bilangan)
print(bilangan_prima) # kosongkan set
bilangan_prima.clear()
print(bilangan_prima)
```

Output:

```
{29, 23, 7, 11, 13}
{29, 23, 7, 11, 13}
29
{23, 7, 11, 13}
set()
```

<code>discard()</code>	<code>remove()</code>	<code>pop()</code>	<code>clear()</code>
Menghapus satu elemen yang disebutkan	Menghapus satu elemen yang disebutkan	Mengambil salah satu dan menghapusnya dari set (tidak tentu)	Menghapus seluruh elemen di dalam set
Tidak ada error	Muncul error jika elemen yang dihapus tidak ada	Error jika set kosong	Tidak ada error

Gambar 12.1: Fungsi-fungsi untuk menghapus anggota dari Set.

Fungsi `pop()` mengambil dan menghapus salah satu anggota Set secara acak, sehingga cocok digunakan untuk memproses isi Set satu per satu tanpa memperhatikan urutan. Sementara itu, `discard()` tidak akan menampilkan error jika anggota yang ingin dihapus tidak ditemukan di dalam Set.

Nilai anggota Set tidak dapat diubah secara langsung. Untuk menggantinya, anggota lama harus dihapus terlebih dahulu, lalu ditambahkan anggota baru dengan nilai yang diinginkan. Contohnya ditunjukkan pada program berikut.

```
#Buat Set dari List
ikan = set(['koi', 'koki', 'kembung', 'salmon'])
print(ikan)
#ganti koi menjadi teri
ikan.remove('koi')
ikan.add('teri')
print(ikan)
```

Output:

```
{'koi', 'koki', 'salmon', 'kembung'}
{'teri', 'salmon', 'kembung', 'koki'}
```

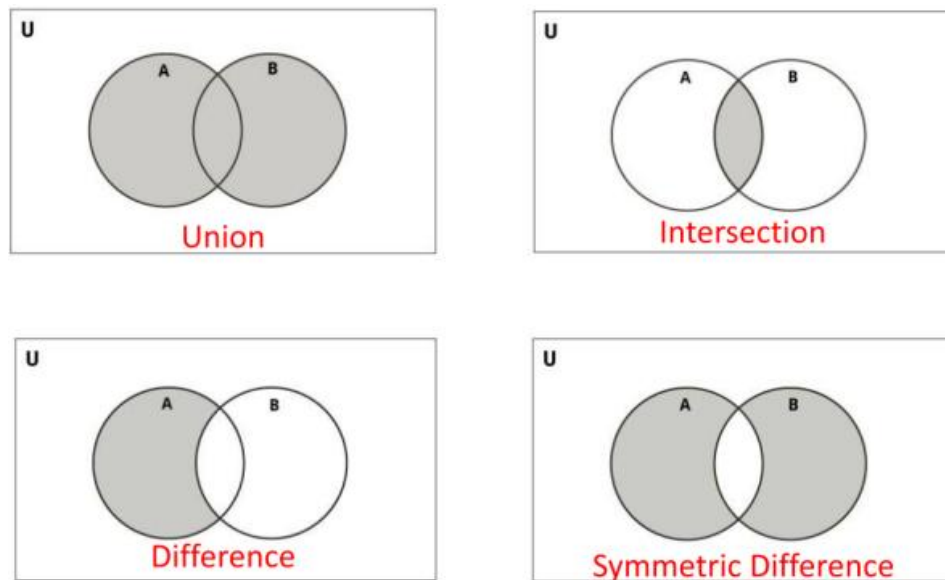
Untuk mengubah nilai "koi" menjadi "teri", pertama-tama harus menghapus anggota "koi" dan kemudian menambahkan anggota baru dengan nilai "teri". Sangat penting untuk diingat bahwa set tidak mengenal posisi atau urutan data, sehingga nilai "koi" tidak ada di output program dan digantikan oleh nilai "teri". Setelah itu, urutan isi Set ikan saat ditampilkan berubah. Urutan anggota dalam Set biasanya akan berubah setiap kali ada penambahan atau penghapusan.

C. Operasi-Operasi pada Set

Operasi-operasi pada Set adalah operasi-operasi pada himpunan. Berikut ini adalah daftar operasi-operasi Set pada Python:

- Operator union dapat menggabungkan dua set menjadi satu dengan menggunakan operator `|` dan fungsi `union()`.

- Operator Intersection. Menghasilkan irisan dari dua Set. Dapat menggunakan operator & maupun fungsi intersection().
- Operator Difference. Menghasilkan Set baru yang merupakan selisih dari dua Set yang dibandingkan. Dapat menggunakan operator - maupun fungsi difference().
- Operator Symmetric Difference. Menghasilkan Set baru yang merupakan jumlah dari dua Set kecuali irisannya. Dapat menggunakan operator maupun fungsi symmetric_difference().



Gambar 12.2: Operasi-operasi pada Set.

➤ Operator Union

```
merek_hp = {'Samsung', 'Apple', 'Xiaomi', 'Sony'}
merek_ac = {'LG', 'Samsung', 'Panasonic', 'Daikin', 'Sony'}
# union dari merek_hp dan merek_ac
gabungan = merek_hp | merek_ac
# bisa juga menggunakan gabungan = merek_hp.union(merek_ac)
print(gabungan)
```

Output:

```
{'Apple', 'Xiaomi', 'Daikin', 'Samsung', 'Sony', 'LG', 'Panasonic'}
```

Anggota dari Set merek_hp dan anggota dari Set merek_ac digabungkan oleh program, menghasilkan Set baru yang disebut gabungan. Ada duplikasi (anggota yang sama), jadi "Samsung" dan "Sony" hanya muncul satu kali di Set gabungan karena anggota harus unik.

➤ Operator Intersection

```
renang = {'siti', 'mail', 'ikhshan', 'upin', 'ipin'}  
tenis = {'joko', 'mail', 'ipin', 'upin', 'tejo'}  
  
# suka renang dan tenis  
renang_tenis = renang & tenis  
  
print(renang_tenis)
```

Output:

```
{'upin', 'mail', 'ipin'}
```

Program akan menghasilkan output dari operasi intersection dari Set Renang dan Tennis, yang berarti anggota yang berada di kedua set, yaitu mail, upin, dan ipin, sekaligus.

➤ Operator Difference

```
#bisa berbahasa english  
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}  
#bisa berbahasa korea  
korean = {'chaewon', 'yeona', 'erika', 'miko'}  
#siapa yang hanya bisa bahasa korea?  
only_korean = korean - english  
print(only_korean)  
#siapa yang hanya bisa bahasa english?  
only_english = english - korean  
print(only_english)
```

Output:

```
{'erika', 'yeona'}  
  
['evan', 'tono', 'takashi', 'desi']
```

Set yang anggotanya adalah selisih dari dua set yang dibandingkan akan dihasilkan oleh operator perbedaan. Pada contoh ini, untuk mendapatkan anggota yang hanya bisa berbahasa

Korea, Anda harus mencari selisih antara set Korea dan set English. Sebaliknya, jika Anda ingin mengetahui siapa saja yang hanya bisa berbahasa Inggris, Anda harus mencari selisih antara set Korea dan set English.

➤ Operator Symmetric Difference

```
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}  
korean = {'chaewon', 'yeona', 'erika', 'miko'}  
  
# hanya bisa bicara satu bahasa saja  
  
one_language = english ^ korean  
  
print(one_language)
```

Operator perbedaan simetrik akan menghasilkan set baru yang merupakan gabungan dari dua set tetapi tidak termasuk irisannya. Contoh berikut menggunakannya untuk mendapatkan siapa saja yang hanya dapat berbicara dalam satu bahasa. Selain itu, biasanya dapat dihasilkan dari operasi berikut:

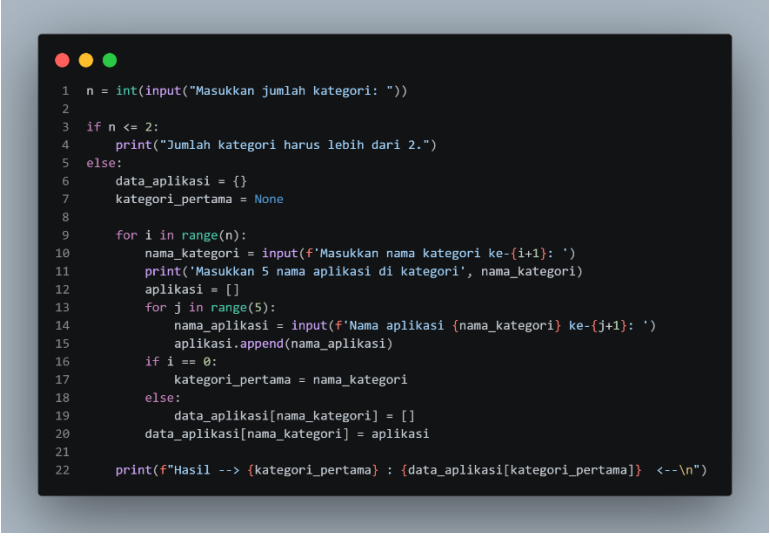
```
one_language = english.union(korean) - english.intersection(korean)  
  
Output:  
  
{'tono', 'desi', 'evan', 'erika', 'takashi', 'yeona'}
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github: <https://github.com/fieliel-star/tugas-prakAlpro-minggu1-71251214.git>

SOAL 1

A screenshot of a terminal window with a dark background and light-colored text. The terminal shows a Python script being executed. The script starts by asking the user for the number of categories. If the user enters a number less than or equal to 2, it prints a message saying the number must be greater than 2. If the user enters a number greater than 2, it proceeds to ask for the names of the categories. For each category, it asks for five application names and stores them in a list. It also stores the first category name in a variable. Finally, it prints the results in a formatted string.

```
1 n = int(input("Masukkan jumlah kategori: "))
2
3 if n <= 2:
4     print("Jumlah kategori harus lebih dari 2.")
5 else:
6     data_aplikasi = {}
7     kategori_pertama = None
8
9     for i in range(n):
10        nama_kategori = input(f'Masukkan nama kategori ke-{i+1}: ')
11        print(f'Masukkan 5 nama aplikasi di kategori', nama_kategori)
12        aplikasi = []
13        for j in range(5):
14            nama_aplikasi = input(f>Nama aplikasi {nama_kategori} ke-{j+1}: ')
15            aplikasi.append(nama_aplikasi)
16        if i == 0:
17            kategori_pertama = nama_kategori
18        else:
19            data_aplikasi[nama_kategori] = []
20            data_aplikasi[nama_kategori] = aplikasi
21
22 print(f"Hasil --> {kategori_pertama} : {data_aplikasi[kategori_pertama]} <--\n")
```

Penjelasan:

Program ini digunakan untuk mendata beberapa kategori aplikasi beserta lima aplikasi pada setiap kategori. Di awal, program meminta pengguna memasukkan jumlah kategori yang diinginkan. Jika jumlah kategori kurang dari atau sama dengan dua, program akan menampilkan pesan bahwa jumlah kategori harus lebih dari dua. Namun jika jumlah kategori sesuai, program akan melanjutkan proses input data.

Selanjutnya, program menggunakan perulangan untuk meminta nama setiap kategori dan lima nama aplikasi yang termasuk di dalam kategori tersebut. Nama-nama aplikasi disimpan ke dalam sebuah list, kemudian list tersebut dimasukkan ke dalam dictionary data_aplikasi. Pada dictionary ini, nama kategori berfungsi sebagai key, sedangkan daftar aplikasinya menjadi value.

Selain itu, program juga menyimpan nama kategori pertama ke dalam variabel kategori_pertama. Setelah seluruh data selesai dimasukkan, program menampilkan kategori pertama beserta daftar aplikasi yang ada di dalamnya. Program ini menunjukkan penggunaan

input, perulangan, list, dan dictionary dalam Python untuk mengelompokkan data aplikasi berdasarkan kategorinya.

Input dan Output:

```
Masukkan jumlah kategori: 3
Masukkan nama kategori ke-1: Game
Masukkan 5 nama aplikasi di kategori Game
Nama aplikasi Game ke-1: PUBG
Nama aplikasi Game ke-2: Free Fire
Nama aplikasi Game ke-3: Mobile Legend
Nama aplikasi Game ke-4: Subway Surf
Nama aplikasi Game ke-5: Roblox
Masukkan nama kategori ke-2: Tools
Masukkan 5 nama aplikasi di kategori Tools
Nama aplikasi Tools ke-1: memo
Nama aplikasi Tools ke-2: RaR
Nama aplikasi Tools ke-3: PDF Reader
Nama aplikasi Tools ke-4: Word Office
Nama aplikasi Tools ke-5: Exel
Masukkan nama kategori ke-3: Sosial Media
Masukkan 5 nama aplikasi di kategori Sosial Media
Nama aplikasi Sosial Media ke-1: Facebook
Nama aplikasi Sosial Media ke-2: Instagram
Nama aplikasi Sosial Media ke-3: X
Nama aplikasi Sosial Media ke-4: TikTok
Nama aplikasi Sosial Media ke-5: Snapchat
Hasil --> Game : ['PUBG', 'Free Fire', 'Mobile Legend', 'Subway Surf', 'Roblox'] <--
```

SOAL 2

```
1 def tampilkan_konversi(asli, terkonversi):
2     print(f'Data sebelum konversi:{asli}',type(asli))
3     print(f'Data setelah konversi:{terkonversi}',type(terkonversi))
4     print()
5
6 def list_ke_set(data_list):
7     set_data = set(data_list)
8     tampilkan_konversi(data_list, set_data)
9     return set_data
10
11 def set_ke_list(data_set):
12     list_data = list(data_set)
13     tampilkan_konversi(data_set, list_data)
14     return list_data
15
16 def tuple_ke_set(data_tuple):
17     set_data = set(data_tuple)
18     tampilkan_konversi(data_tuple, set_data)
19     return set_data
20
21 def set_ke_tuple(data_set):
22     tuple_data = tuple(data_set)
23     tampilkan_konversi(data_set, tuple_data)
24     return tuple_data
25
26 data_list = [1, 2, 3, 4, 5]
27 data_set = {5, 6, 7, 8, 9}
28 data_tuple = (10, 11, 12, 13, 14)
29
30 hasil_set = list_ke_set(data_list)
31 hasil_list = set_ke_list(data_set)
32 hasil_set2 = tuple_ke_set(data_tuple)
33 hasil_tuple = set_ke_tuple(data_set)
```

Penjelasan:

Program ini dibuat untuk mendemonstrasikan proses konversi beberapa tipe data di Python, yaitu dari list ke set, set ke list, tuple ke set, dan set ke tuple. Di awal program terdapat fungsi `tampilkan_konversi()` yang digunakan untuk menampilkan data sebelum dan sesudah dikonversi beserta tipe datanya. Dengan fungsi ini, hasil konversi menjadi lebih mudah dipahami karena pengguna dapat melihat perubahan tipe data secara langsung.

Selanjutnya, program memiliki beberapa fungsi khusus seperti `list_ke_set()`, `set_ke_list()`, `tuple_ke_set()`, dan `set_ke_tuple()`. Masing-masing fungsi bertugas melakukan konversi menggunakan fungsi bawaan Python seperti `set()`, `list()`, dan `tuple()`. Setelah data dikonversi, fungsi `tampilkan_konversi()` dipanggil untuk menampilkan hasilnya.

Pada bagian akhir, program membuat contoh data berupa list, set, dan tuple, kemudian menjalankan semua fungsi konversi tersebut. Program ini menunjukkan bagaimana Python dapat mengubah tipe data dengan mudah dan tetap mempertahankan isi data yang ada.

Output:

```
Data sebelum konversi:[1, 2, 3, 4, 5] <class 'list'>
Data setelah konversi:{1, 2, 3, 4, 5} <class 'set'>

Data sebelum konversi:{5, 6, 7, 8, 9} <class 'set'>
Data setelah konversi:[5, 6, 7, 8, 9] <class 'list'>

Data sebelum konversi:(10, 11, 12, 13, 14) <class 'tuple'>
Data setelah konversi:{10, 11, 12, 13, 14} <class 'set'>

Data sebelum konversi:{5, 6, 7, 8, 9} <class 'set'>
Data setelah konversi:(5, 6, 7, 8, 9) <class 'tuple'>
```

SOAL 3

```
1 def cek_kata_sama(txt1, txt2):
2     try:
3         with open(txt1, 'r') as file1, open(txt2, 'r') as file2:
4             kata1 = set(file1.read().lower().split())
5             kata2 = set(file2.read().lower().split())
6
7         except:
8             print(f"Error: File tidak ditemukan, Coba kembali.")
9             return
10
11     kata_sama = list(kata1.intersection(kata2))
12
13     if kata_sama:
14         print("Kata-kata yang sama di kedua file:")
15         print(kata_sama)
16     else:
17         print("Tidak ada kata yang sama di kedua file.")
18
19     jumlah_kata_sama = len(kata_sama)
20     return jumlah_kata_sama
21
22 txt1 = input("Masukkan nama file pertama: ")
23 txt2 = input("Masukkan nama file kedua: ")
24
25 jumlah_kata = cek_kata_sama(txt1, txt2)
26 if jumlah_kata is not None:
27     print(f"(Jumlah kata yang sama di kedua file adalah: {jumlah_kata})\n")
```

Penjelasan:

Program ini digunakan untuk mencari kata-kata yang sama dari dua file teks. Pertama, program membuat fungsi `cek_kata_sama()` yang menerima dua nama file sebagai parameter. Di dalam fungsi tersebut, program membuka kedua file menggunakan `with open()`, lalu membaca seluruh isi file, mengubah semua huruf menjadi kecil dengan `lower()`, dan memisahkan setiap kata menggunakan `split()`. Setelah itu, kata-kata dari masing-masing file disimpan ke dalam tipe data set agar tidak ada kata yang duplikat.

Program juga menggunakan try-except untuk menangani error jika file yang dimasukkan tidak ditemukan. Jika terjadi kesalahan, program akan menampilkan pesan error dan proses dihentikan. Selanjutnya, program menggunakan `intersection()` untuk mencari kata-kata yang muncul di kedua file. Hasilnya diubah menjadi list dan ditampilkan ke layar. Jika tidak ada kata yang sama, program akan menampilkan pesan bahwa tidak ditemukan kata yang sama.

Di bagian akhir, program meminta pengguna memasukkan nama dua file teks, kemudian memanggil fungsi `cek_kata_sama()` untuk membandingkan isi kedua file tersebut. Selain menampilkan daftar kata yang sama, program juga menghitung jumlah katanya menggunakan fungsi `len()`.

Output

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel\minggu14> python -u "d:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel\minggu14\tempcodekun
nerFile.py"
Masukkan nama file pertama: cerita1.txt
Masukkan nama file kedua: cerita2.txt
Kata-kata yang sama di kedua file:
['seorang', 'dia', 'perdamaian', 'militer.', 'menunjukkan', 'pria', 'tugasnya', 'dari', 'dihadapkan', 'dan', 'melindungi', 'semangat', 'yang', 'menyerah', 'luan', 'penuh', 'demi'
, 'siapa', 'adalah', 'perang', 'langkahnya', 'telah', 'bernama', 'kesulitan', 'sebagai', 'setia', 'keberanian', 'wajib', 'di', 'medan', 'untuk', 'selama', 'panggilan', 'tantangan'
, 'pada', 'biasa', 'meskipun', 'ketangguhan', 'dengan', 'tak', 'satu', 'tidak', 'setiap', 'keadilan', 'mempertahankan', 'pernah', 'tetap', 'bergabung', 'memberikan', 'martabat
']
(Jumlah kata yang sama di kedua file adalah: 50)
```